

MICROBOT ECOSYSTEM

Documento 04 dei 12

Firmware & Communication Protocol Specification

Specifica firmware, protocollo JSON, stati, comandi, heartbeat, telemetria, errori e mapping dashboard-hardware.

Campo	Valore
Progetto	MicroBot Ecosystem
Documento	04 - Firmware & Communication Protocol
Versione	v0.1 - prototype specification
Obiettivo	Definire il linguaggio operativo comune tra PC Controller, ESP32 Master e sei nodi MicroBot.
Ambito	Firmware Master, firmware nodi, protocollo JSON, stati, comandi, heartbeat, telemetria, error handling e test.
Output pratico	Base tecnica per scrivere il codice del Master, dei nodi e del bridge con dashboard PC.

1. Scopo del documento

Questo documento definisce il quarto blocco tecnico della documentazione MicroBot: il firmware e il protocollo di comunicazione. Dopo aver stabilito l'architettura hardware, il passaggio successivo consiste nel dare al sistema un linguaggio comune. Il PC Controller deve poter inviare comandi al Master; il Master deve poter inoltrare comandi ai nodi; ogni nodo deve rispondere con uno stato coerente; la dashboard deve poter trasformare questi dati in visualizzazione, telemetria e simulazione.

La specifica non descrive ancora il codice finale riga per riga. Definisce invece l'architettura logica che il codice dovrà rispettare: nomi dei nodi, stati, comandi, formato dei messaggi, timeout, heartbeat, errori, sicurezza, comportamento minimo atteso e criteri di test.

Domanda	Risposta tecnica
Chi comanda il sistema?	Il PC Controller invia comandi al Master ESP32; il Master coordina i nodi.
Come parlano i componenti?	Per la v0.1 si usa JSON semplificato su USB seriale; poi ESP-NOW; in futuro WebSocket/Wi-Fi.
Cosa deve fare ogni nodo?	Rispondere a PING/STATUS, eseguire comandi specifici e inviare telemetria essenziale.
Come si evita il caos?	Ogni nodo ha ID, stato, comandi ammessi, risposta standard, heartbeat e timeout.
Come si integra la simulazione?	La dashboard mappa ogni risposta fisica in un aggiornamento dello stato virtuale.

2. Architettura logica firmware

L'architettura firmware MicroBot v0.1 è organizzata su tre livelli. Il primo livello è il PC Controller, cioè l'interfaccia operativa dell'utente. Il secondo livello è il Master ESP32, che riceve comandi, gestisce lo stato globale, invia richieste ai nodi e riceve risposte. Il terzo livello è formato dai sei nodi specializzati: LED State, Proximity Safety, Magnetic Docking, Motion Actuator, Telemetry Sensor e Vision Camera.

```
PC_CONTROLLER / DASHBOARD
|
| USB Serial v0.1 / WebSocket future
v
NODE_00_MASTER
|
| Serial debug v0.1 / ESP-NOW v0.2 / Wi-Fi future
v
NODE_01_LED_STATE
NODE_02_PROXIMITY_SAFETY
NODE_03_MAGNETIC_DOCKING
NODE_04_MOTION_ACTUATOR
NODE_05_TELEMETRY_SENSOR
NODE_06_VISION_CAMERA
```

Livello	Responsabilità	Firmware richiesto
PC Controller	Invio comandi, visualizzazione, log, simulazione e dashboard utente.	JavaScript/Web Serial o server bridge.
Master ESP32	Coordinamento, routing, scan nodi, heartbeat, error handling e inoltra telemetria.	Firmware Master con parser, state manager e router.
Nodi MicroBot	Esecuzione funzione locale e risposta al Master.	Firmware dedicato per ogni nodo con schema comune.

3. Identità dei nodi

Ogni unità del sistema deve avere un identificativo stabile. Gli ID non sono semplici etichette: sono indirizzi logici usati dal protocollo, dalla dashboard, dalla simulazione e dai log. Per la v0.1 si usano nomi testuali leggibili, più facili da debuggare. In futuro potranno essere aggiunti ID numerici e MAC address ESP-NOW.

ID logico	Nome	Ruolo firmware	Payload principale
NODE_00_MASTER	Master controller	Router, scheduler, heartbeat manager, bridge PC.	node_table, system_state, error_log
NODE_01_LED_STATE	LED State Node	Gestione stato visivo e pattern.	mode, color, pattern
NODE_02_PROXIMITY_SAFETY	Proximity / Safety Node	Lettura distanza e stato sicurezza.	distance_mm, threshold_mm, safety_state
NODE_03_MAGNETIC_DOCKING	Magnetic Docking Node	Rilevamento magnete/docking.	magnetic_level, docked, hall_raw
NODE_04_MOTION_ACTUATOR	Motion / Actuator Node	Servo, motore vibrazione, stop sicuro.	servo_angle, motor_state, motion_state
NODE_05_TELEMETRY_SENSOR	Telemetry / Sensor Node	IMU, temperatura, display, stato interno.	accel, gyro, temperature, display_state
NODE_06_VISION_CAMERA	Vision / Camera Node	Camera status, capture, stream info.	camera_state, streaming, ip, frame_status

4. Livelli di trasporto

Il protocollo MicroBot deve poter sopravvivere a più modalità di trasporto. La struttura dei messaggi rimane uguale; cambia solo il canale fisico/logico usato per inviarli. Questo permette di iniziare in modo semplice con USB seriale e poi passare a ESP-NOW o Wi-Fi senza riscrivere tutta la logica applicativa.

Fase	Trasporto	Uso	Vantaggi	Limiti
v0.1	USB Serial	PC -> Master e debug diretto.	Semplice, stabile, facile da debuggare.	Troppi cavi se tutti i nodi sono collegati.
v0.2	ESP-NOW	Master -> nodi wireless per comandi e stati.	Coerente con swarm, bassa latenza, non richiede router.	Richiede gestione MAC address e debug più attento.
v0.3	Wi-Fi / UDP	Controllo rete locale e dashboard avanzata.	Buono per rete, log, API e web interface.	Latenza maggiore e dipendenza da rete.
v0.4+	WebSocket / API	Dashboard, plugin, notifiche push, telemetria realtime.	Architettura moderna e scalabile.	Richiede server/bridge più strutturato.

5. Formato generale dei messaggi JSON

Per la v0.1 il formato consigliato è JSON line-based: ogni messaggio è una riga JSON terminata da newline. Questo rende semplice il parsing tramite Serial Monitor, Web Serial API e debug manuale.

```
{
  "from": "PC_CONTROLLER",
  "to": "NODE_01_LED_STATE",
  "cmd": "SET_MODE",
  "value": "ACTIVE",
  "request_id": "REQ_0001",
  "timestamp": 123456
}
```

5.1 Campi obbligatori

Campo	Tipo	Obbligatorio	Descrizione
from	string	si	Identificativo del mittente: PC_CONTROLLER, NODE_00_MASTER o nodo specifico.
to	string	si	Destinatario logico del messaggio.
cmd	string	si	Comando richiesto. Deve appartenere alla lista globale o alla lista specifica del nodo.
timestamp	number	si	Tempo locale in millisecondi. Non deve essere usato come orologio assoluto nella v0.1.
request_id	string	consigliato	ID univoco per collegare comando e risposta.

value	any	opzionale	Valore semplice del comando, se basta un solo parametro.
payload	object	opzionale	Oggetto complesso con parametri multipli.

5.2 Risposta standard del nodo

```
{
  "from": "NODE_01_LED_STATE",
  "to": "NODE_00_MASTER",
  "type": "RESPONSE",
  "request_id": "REQ_0001",
  "status": "OK",
  "mode": "ACTIVE",
  "battery": 100,
  "error": "NONE",
  "timestamp": 123490,
  "payload": {
    "color": "GREEN",
    "pattern": "SOLID"
  }
}
```

6. Comandi globali

I comandi globali devono essere implementati da tutti i nodi. Questa scelta permette al Master di trattare i nodi in modo uniforme durante scan, diagnostica, emergenza e recupero errori.

Comando	Destinatari	Descrizione	Risposta attesa
PING	tutti	Verifica se il nodo è online.	PONG/OK con timestamp e node_id.
STATUS	tutti	Richiede stato generale del nodo.	Pacchetto stato con mode, error e batteria simulata/reale.
SET_MODE	tutti	Imposta una modalità operativa.	OK se modalità valida, ERROR se non supportata.
STOP	tutti	Ferma attuatori o porta il nodo in stato sicuro.	OK e stato SAFE_STOP/IDLE.
RESET	tutti	Reset logico dello stato firmware senza riavvio fisico.	OK e stato IDLE/READY.
CALIBRATE	nodi con sensori	Avvia calibrazione locale del sensore o dell'attuatore.	OK/CALIBRATING, poi CALIBRATED o ERROR.
HELP	Master / debug	Restituisce lista comandi supportati.	Elenco comandi o schema breve.

7. Comandi specifici dei nodi

Nodo	Comandi specifici	Output atteso
NODE_01_LED_STATE	SET_COLOR, SET_PATTERN, SET_SWARM_STATE	Aggiornamento colore, pattern e stato swarm simbolico.
NODE_02_PROXIMITY_SAFETY	READ_DISTANCE, SET_DISTANCE_THRESHOLD, ENABLE_ALARM, DISABLE_ALARM	Distanza, soglia, stato SAFE/WARNING/DANGER.
NODE_03_MAGNETIC_DOCKING	READ_DOCKING, SET_DOCKING_LED, CALIBRATE_HALL	Stato magnete, hall_raw, docked true/false.
NODE_04_MOTION_ACTUATOR	SET_SERVO_ANGLE, SWEEP_SERVO, MOTOR_FORWARD, MOTOR_BACKWARD, MOTOR_STOP	Angolo servo, stato motore, motion_state.
NODE_05_TELEMETRY_SENSOR	READ_IMU, DISPLAY_TEXT, DISPLAY_STATUS, READ_TELEMETRY	Accelerazione, giroscopio, temperatura, stato display.
NODE_06_VISION_CAMERA	CAPTURE, STREAM_ON, STREAM_OFF, READ_CAMERA_STATUS	Camera ready, streaming true/false, IP o endpoint.

8. Macchine a stati

Ogni firmware deve usare una macchina a stati semplice. La macchina a stati evita comportamenti ambigui e permette di decidere cosa può fare un nodo in ogni momento. Un comando può essere accettato o rifiutato in base allo stato corrente.

Categoria	Stati consigliati	Uso
Master	MASTER_BOOT, MASTER_SCANNING, MASTER_READY, MASTER_RUNNING, MASTER_WARNING, MASTER_ERROR, MASTER_STOPPED	Coordinamento globale e gestione nodi.
Nodo base	BOOT, IDLE, ACTIVE, CALIBRATING, LOW_POWER, ERROR, SAFE_STOP	Struttura comune per tutti i nodi.
NODE_02 safety	SAFE, WARNING, DANGER, SENSOR_ERROR	Interpretazione del valore distanza.
NODE_03 docking	NOT_DOCKED, MAGNET_NEAR, DOCKED, DOCKING_ERROR	Interpretazione del sensore Hall e dell'aggancio.
NODE_04 motion	MOTION_IDLE, MOTION_ACTIVE, MOTION_SWEEP, MOTION_ERROR	Controllo attuatori e stop sicuro.
NODE_06 camera	CAM_BOOT, CAM_READY, STREAMING, CAPTURED, CAM_ERROR, LOW_POWER	Controllo camera e streaming.

9. Heartbeat e monitoraggio online/offline

Il sistema deve sapere quali nodi sono online. Per questo il Master invia periodicamente PING o STATUS e aggiorna una node table. Ogni nodo deve rispondere entro un timeout definito. Se il timeout viene superato, il nodo passa prima in stato WARNING e poi OFFLINE.

Parametro	Valore v0.1 consigliato	Note
heartbeat_interval_ms	1000 ms	Il Master richiede stato una volta al secondo.
node_timeout_ms	3000 ms	Dopo 3 secondi senza risposta il nodo è sospetto.
offline_timeout_ms	5000 ms	Dopo 5 secondi senza risposta il nodo è OFFLINE.
status_report_interval_ms	1000-2000 ms	I nodi possono inviare stato periodico in modalità avanzata.
dashboard_log_rate	limitare a eventi importanti	Evitare log infinito o unreadable.

```
if current_time - node.last_seen > node_timeout_ms:  
    node.state = "WARNING"
```

```
if current_time - node.last_seen > offline_timeout_ms:  
    node.state = "OFFLINE"  
    send_event_to_dashboard("NODE_OFFLINE", node.id)
```

10. Error handling

Gli errori devono essere standardizzati. Un errore non deve essere solo una stringa casuale nel Serial Monitor: deve essere leggibile dal Master e dalla dashboard. Ogni risposta può contenere error = NONE oppure un codice preciso.

Codice errore	Significato	Azione consigliata
NONE	Nessun errore.	Continuare.
UNKNOWN_CMD	Comando non riconosciuto.	Mostrare log e ignorare comando.
INVALID_PAYLOAD	Parametri mancanti o tipo sbagliato.	Rispondere ERROR e non eseguire.
UNSUPPORTED_MODE	Modalità non supportata da quel nodo.	Non cambiare stato.
SENSOR_READ_FAIL	Lettura sensore fallita.	Passare a WARNING o ERROR.
ACTUATOR_FAIL	Attuatore non inizializzato o bloccato.	Eseguire STOP locale.
LOW_POWER	Batteria bassa o simulazione batteria sotto soglia.	Ridurre attuazione o fermare nodo.
OVER_TEMP	Temperatura troppo alta o simulata alta.	Spegnere bobina/motore.
TIMEOUT	Nodo non risponde.	Master marca WARNING/OFFLINE.
EMERGENCY_STOP	Stop globale attivato.	Tutti i nodi in SAFE_STOP.

11. Sicurezza firmware

La sicurezza firmware non riguarda solo cybersecurity, ma soprattutto evitare comportamenti fisici pericolosi. Servo, motori, bobine ed elettromagneti devono avere timeout e limiti di duty cycle. Il comando STOP deve avere priorità assoluta su ogni altro comando.

Regola	Applicazione
STOP prioritario	Se arriva STOP, interrompere immediatamente pattern, motore, servo sweep o bobina.
Timeout attuatori	Ogni attuazione deve avere durata massima. Nessun motore deve restare acceso per errore.
Fail-safe locale	Se il nodo non riceve heartbeat dal Master per N secondi, passa in SAFE_STOP.
No blocking loop	Usare millis() e update() non bloccante; evitare delay lunghi.
Validazione payload	Non accettare angoli servo fuori range o soglie impossibili.
Logging errori	Ogni errore critico deve essere comunicato a Master e dashboard.

12. Mapping dashboard - firmware - simulazione

Il protocollo non serve solo a far funzionare i circuiti. Serve anche a tenere allineata la simulazione. Ogni evento fisico deve avere una conseguenza virtuale e ogni comando virtuale deve poter generare un comando fisico.

Azione dashboard	Comando firmware	Risposta nodo	Aggiornamento simulazione
Premi ACTIVE su NODE_01	SET_MODE ACTIVE	mode=ACTIVE, color=GREEN	Nodo virtuale verde/attivo.
Leggi distanza NODE_02	READ_DISTANCE	distance_mm=180, safety=WARNING	Zona rossa/arancione intorno al bot.
Avvicina magnete NODE_03	READ_DOCKING o evento periodico	docked=true	Due nodi virtuali si agganciano.
Muovi slider servo NODE_04	SET_SERVO_ANGLE 90	servo_angle=90	Rotazione del microbot virtuale.
Leggi IMU NODE_05	READ_IMU	accel/gyro payload	Orientamento/inclinazione nella scena.
Avvia camera NODE_06	STREAM_ON	streaming=true, ip=...	Pannello camera attivo.

13. Esempi completi di messaggi

13.1 Scan nodi

```
PC -> MASTER
{"from":"PC_CONTROLLER","to":"NODE_00_MASTER","cmd":"SCAN_NODES","request_id":"REQ_SCAN_001","timestamp":1000}

MASTER -> PC
{
  "from":"NODE_00_MASTER",
  "to":"PC_CONTROLLER",
  "type":"NODE_TABLE",
  "status":"OK",
  "nodes":[
    {"id":"NODE_01_LED_STATE","online":true,"last_seen":1020},
    {"id":"NODE_02_PROXIMITY_SAFETY","online":true,"last_seen":1032},
    {"id":"NODE_03_MAGNETIC_DOCKING","online":false,"error":"TIMEOUT"}
  ]
}
```

13.2 Comando LED Node

```
MASTER -> NODE_01_LED_STATE
{"from":"NODE_00_MASTER","to":"NODE_01_LED_STATE","cmd":"SET_COLOR","value":"GREEN","request_id":"REQ_LED_010","timestamp":2000}

NODE_01_LED_STATE -> MASTER
{"from":"NODE_01_LED_STATE","to":"NODE_00_MASTER","status":"OK","mode":"ACTIVE","error":"NONE","payload":{"color":"GREEN"},"request_id":"REQ_LED_010","timestamp":2015}
```

13.3 Telemetria IMU

```
NODE_05_TELEMETRY_SENSOR -> MASTER
{
  "from":"NODE_05_TELEMETRY_SENSOR",
  "to":"NODE_00_MASTER",
  "type":"TELEMETRY",
  "status":"OK",
  "payload":{
    "accel":{"x":0.02,"y":0.01,"z":9.81},
    "gyro":{"x":0.2,"y":0.0,"z":0.1},
    "temperature":26.4
  },
  "timestamp":5022
}
```

14. Firmware Master: struttura consigliata

Il Master deve essere scritto come programma non bloccante. Non deve restare fermo ad aspettare un nodo. Deve leggere seriale, aggiornare heartbeat, gestire la node table, inoltrare comandi, ricevere risposte e inviare eventi al PC.

```
setup():
  init_serial()
  init_oled_optional()
  init_transport()
  init_node_table()
  set_master_state(MASTER_BOOT)

loop():
  read_pc_serial()
  parse_incoming_command()
  route_command_if_valid()
  update_heartbeat()
  receive_node_messages()
  update_node_table()
  send_dashboard_events()
  handle_errors()
```

Modulo firmware Master	Responsabilita
serial_bridge	Riceve e invia messaggi verso PC Controller.
command_parser	Valida JSON e traduce comandi in azioni.
node_router	Inoltra comandi verso nodo corretto.
node_table	Mantiene stato online/offline, last_seen, mode, error.
heartbeat_manager	Invia PING/STATUS periodici.
safety_manager	Gestisce STOP globale, timeout e fault.
telemetry_forwarder	Inoltra dati dei nodi alla dashboard.

15. Firmware nodo: struttura comune

Ogni nodo deve condividere una struttura base. La parte specifica cambia in base al sensore o attuatore, ma il ciclo firmware deve restare comune: inizializzazione, lettura messaggi, validazione comando, esecuzione, update non bloccante, risposta e sicurezza locale.

```
setup():
    init_node_id()
    init_serial_or_transport()
    init_hardware()
    set_state(BOOT)
    send_boot_message()

loop():
    read_master_command()
    if command_available:
        validate_command()
        execute_command()
        send_response()

    update_local_state()
    update_sensors_or_actuators()
    check_local_timeout()
    send_periodic_status_if_needed()
```

16. Specifiche firmware per singolo nodo

NODE_01_LED_STATE

Voce	Specifica
Descrizione	Gestisce LED RGB/NeoPixel, pattern e stato simbolico dello swarm.
Init minimo	Inizializzare pin LED, colore base e modalita IDLE.
Comandi globali	PING, STATUS, SET_MODE, STOP, RESET.
Safety	Spegnere o portare a colore errore in caso di comando non valido.
Risposta minima	status, mode, error, timestamp, payload specifico.

NODE_02_PROXIMITY_SAFETY

Voce	Specifica
Descrizione	Legge distanza e genera SAFE/WARNING/DANGER.
Init minimo	Inizializzare sensore ToF o HC-SR04 e soglia default.
Comandi globali	PING, STATUS, SET_MODE, STOP, RESET.
Safety	Se lettura fallisce, passare SENSOR_ERROR e avvisare il Master.
Risposta minima	status, mode, error, timestamp, payload specifico.

NODE_03_MAGNETIC_DOCKING

Voce	Specifica
Descrizione	Rileva magneti e stato docking tramite Hall sensor.
Init minimo	Calibrare baseline Hall e soglia magneti.
Comandi globali	PING, STATUS, SET_MODE, STOP, RESET.
Safety	Non attivare elettromagneti senza timeout e limite duty cycle.
Risposta minima	status, mode, error, timestamp, payload specifico.

NODE_04_MOTION_ACTUATOR

Voce	Specifica
Descrizione	Controlla servo e motore vibrazione.
Init minimo	Impostare servo a posizione neutra e motore spento.
Comandi globali	PING, STATUS, SET_MODE, STOP, RESET.
Safety	STOP immediato, range servo 0-180, timeout sweep.
Risposta minima	status, mode, error, timestamp, payload specifico.

NODE_05_TELEMETRY_SENSOR

Voce	Specifica
Descrizione	Raccoglie IMU, temperatura e mostra dati su OLED.
Init minimo	Inizializzare I2C, MPU6050 e display.
Comandi globali	PING, STATUS, SET_MODE, STOP, RESET.
Safety	Se IMU non risponde, mantenere nodo vivo ma telemetry_error=true.
Risposta minima	status, mode, error, timestamp, payload specifico.

NODE_06_VISION_CAMERA

Voce	Specifica
Descrizione	Gestisce stato camera, capture e streaming.
Init minimo	Inizializzare camera, rete o endpoint stream.
Comandi globali	PING, STATUS, SET_MODE, STOP, RESET.
Safety	Se stream non parte, rispondere CAM_ERROR senza bloccare il sistema.
Risposta minima	status, mode, error, timestamp, payload specifico.

17. Roadmap implementativa firmware

Fase	Obiettivo	Risultato atteso
1	Scrivere parser JSON seriale su Master.	PC invia comando e Master risponde.
2	Implementare node table e scan simulato.	Dashboard vede elenco nodi anche se simulati.
3	Implementare NODE_01 reale.	PC -> Master -> NODE_01 -> risposta -> dashboard.
4	Aggiungere NODE_02 e NODE_03.	Distanza e docking entrano nella dashboard/simulazione.
5	Aggiungere NODE_04 e NODE_05.	Movimento e telemetria funzionano.
6	Aggiungere NODE_06.	Camera status e stream separato.
7	Migrare Master-nodi a ESP-NOW.	Meno cavi, architettura piu swarm-like.
8	Integrare WebSocket/API.	Dashboard realtime piu robusta.

18. Test di validazione

Test	Procedura	Criterio di successo
T01 JSON parser	Inviare JSON valido e non valido al Master.	Valido accettato; non valido produce INVALID_PAYLOAD.
T02 PING	Inviare PING a ogni nodo.	Ogni nodo risponde entro timeout.
T03 STATUS	Richiedere STATUS a tutti i nodi.	Risposta contiene mode, status, error e timestamp.
T04 STOP	Avviare attuatore/pattern e inviare STOP.	Il nodo si ferma immediatamente.
T05 Offline	Scollegare un nodo.	Master passa WARNING poi OFFLINE.
T06 Telemetry	Muovere NODE_05.	Dashboard riceve dati IMU e li logga.
T07 Docking	Avvicinare magnete a NODE_03.	docked=true e simulazione aggiornata.
T08 Camera	Avviare STREAM_ON.	NODE_06 comunica CAM_READY/STREAMING o errore leggibile.

19. Struttura repository firmware consigliata

```
microbot_firmware/  
  README.md  
  protocol/  
    protocol_v0_1.md  
    message_schema.json  
    error_codes.md
```

```
master/  
  master_firmware.ino  
  node_table.h  
  command_router.h  
  heartbeat_manager.h  
nodes/  
  node_01_led_state/  
  node_02_proximity_safety/  
  node_03_magnetic_docking/  
  node_04_motion_actuator/  
  node_05_telemetry_sensor/  
  node_06_vision_camera/  
shared/  
  microbot_ids.h  
  microbot_protocol.h  
  microbot_states.h  
  microbot_errors.h  
tests/  
  serial_test_messages.jsonl  
  validation_checklist.md
```

20. Conclusione operativa

Questo documento stabilisce il linguaggio minimo con cui MicroBot può diventare un sistema reale. La priorità non è scrivere subito un firmware perfetto, ma costruire una catena funzionante e verificabile: PC Controller -> Master -> Nodo -> Risposta -> Dashboard -> Simulazione. Quando questa catena funziona con NODE_01, il resto dei nodi diventa un'estensione ordinata dello stesso schema.

La prossima fase documentale, il Documento 05, dovrà specificare il PC Controller e la Dashboard: struttura dei file web, Web Serial API, pannelli UI, log, mapping con il dimension engine e visualizzazione degli stati dei nodi.